Manuscript revision comparison

Bangla Sign Language Gesture Recognition

Nazmul Hassan

Added / replaced ~~Removed / replaced~~

Comparison

Preserved ScienceOpen version 1 compared with the clean revised manuscript. This file is for the tracked-changes upload field; the clean revised manuscript belongs in the main content field.

How to read the markup

Part A contains the revised manuscript. Blue underlining marks added or replaced words. Part B contains the original manuscript. Red strikethrough marks removed or replaced words. Unmarked text matched between versions. Page numbers within each part belong to that source manuscript.

Scope of the comparison

The revision substantially rewrites and extends the original text, so extensive markup is expected. This is an automated comparison of text extracted from the two PDFs, not a native word-processor change log. Reordered text, line-end hyphenation and extraction differences can appear as removal plus addition. Figures and tables remain visible in their respective versions; graphical content is not compared pixel by pixel.

Document integrity

The source PDFs are unchanged. The comparison preserves the original claims for inspection and does not treat them as results of the additional experiments. A separate revision summary is supplied in the ScienceOpen submission guide.

Contents

Part A: revised manuscript, 10 pages
Part B: original ScienceOpen preprint, 8 pages
Original DOI: 10.14293/S2199-1006.1.SOR-.PPUF56Q.v1

Bangla Sign Language Gesture Recognition: Static Image Classification and Bounded Sentence Recognition

Nazmul Hassan
North South University, Dhaka, Bangladesh
ORCID: 0000-0001-6116-017X
iam.najib@yahoo.com

Preprint

Abstract

This study evaluates static Bangla hand-sign classification and exploratory recognition of a bounded sentence vocabulary, extending an undergraduate thesis system. The surviving Python architecture is retrained once, while a frozen MobileNetV2 classifier provides a modest comparison. A separate experiment uses the openly licensed KU-BdSL USLD collection; its grouped label space is kept distinct from the historical dataset. Evaluation uses fixed seeded partitions, content audits, class metrics, confusion matrices and prediction-level evidence. The historical CNN obtains 3.38% on 148 held-out images; the frozen comparison obtains 52.70% on the same split. The separate public-data model obtains 99.58% on 240 held-out images. These image-level tests do not establish unseen-signer performance. The software accepts images and recorded files as well as cameras, and a fixture-video test verifies the pipeline without camera hardware. A separate exploratory ten-sentence classifier recognises 6 of ten supported test clips and rejects 8 of ten background clips. The results support constrained recognition experiments; continuous translation and performance with new signers require further evaluation.

1. Introduction

Accessible communication tools require reliable evidence as well as working interfaces. A recogniser for a small set of hand configurations may support a constrained input experiment, but it does not capture the movement, context and grammar of a natural sign language. We evaluate static class prediction and a separate, bounded sentence-recognition task.

The original system was described in a ScienceOpen preprint [1]. Differences between its methods narrative and the surviving implementation motivate an explicit reconstruction of the executable CNN. This study contributes audited data partitions, reproducible model comparisons, class-level error analysis and an exploratory temporal model. The original preprint is retained unchanged; the additional experiments are reported separately.

2. Related work and scope

The KU-BdSL authors provide an openly available collection designed for Bengali consonant recognition [2]. The accompanying versioned dataset offers an explicit licence and documented collection conditions [3]. It is useful here because its class layout can be downloaded, hashed and audited without treating an undocumented image bundle as an independent benchmark.

MobileNetV2 introduces inverted residuals and linear bottlenecks for efficient vision models [4]. Here it is used only as an established, frozen ImageNet feature extractor. A small new classifier is learned on top. Its frozen-backbone configuration provides a controlled, computationally economical transfer-learning baseline.

The historical and public collections have different image distributions and label spaces. We therefore compare architectures within the historical split and report the public-data experiment separately; cross-dataset score differences cannot isolate architectural improvement.

3. Data and provenance

The historical repository contains 1,005 labelled images in each of three representations: Gray, Resized and Otsu_again. They are not independent collections. The re-run uses Gray only; the other copies are excluded from training and evaluation. Its 36 numeric folder labels inherit a provisional display mapping from the original interface. The duplicate vowel for class 6 is assigned a provisional display correction; expert verification remains outstanding. Quantitative evaluation retains the original numeric identifiers.

The historical paper describes constrained camera capture and hand cropping, but the repository supplies no verified signer identifiers, capture-session identifiers or participant-consent documentation. This revision therefore does not assert a verified count of individual historical signers. It also does not infer the collection of depth data merely from a prose claim when the available image pipeline does not expose such measurements.

The new experiment downloads KU-BdSL version 4, USLD only. The publisher describes smartphone images from 39 participants and 30 grouped consonant classes. Several folder labels represent multiple written characters with a shared sign class. Their decimal Unicode identifiers are preserved, including grouped identifiers; they are not silently expanded into separate targets. The measured inventory is recorded alongside the download manifest.

4. Split protocol and dataset audit

Before splitting, the code decodes every candidate image, applies the exact evaluation preprocessing, and hashes the resulting float32 tensor with SHA256. Identical inputs are grouped together. A group with conflicting labels is excluded and logged. A group with one label contributes one representative, with duplicate paths retained in the manifest. Unreadable inputs are excluded explicitly. This removes exact input duplicates, but it cannot guarantee independence between visually similar captures.

Each class is shuffled with seed 2026. Test and validation each receive the nearest integer to 15% of that class, with at least one example; the remainder is used for training. All records retain paths, class IDs, content hashes and partition membership. The historical CNN and its transfer baseline use the same saved manifest; the trainer validates the current content against it before reuse.

The historical audit finds 1005 unique usable images, split into 709 training, 148 validation and 148 test images. The public-data audit finds 1500 unique usable images, split into 1020 training, 240 validation and 240 test images. Exact duplicate and conflict details are in the machine-readable inventories. These counts describe decoded, usable inputs, not estimates inferred from a nominal dataset size.

No image in the test manifest is used for fitting or validation. Augmentation is applied only to

historical training batches. The fixed 12-epoch schedule is specified in advance; there is no test-based checkpoint selection or search over random seeds. Examples in the showcase are selected after evaluation for illustration and never used to estimate accuracy.

The partition is image-level. Although the public dataset documents multiple participants, the downloaded folder labels do not provide signer IDs suitable for a verified signer-disjoint split. Shared signers, backgrounds or sessions may therefore cross partitions. Near duplicates may also survive the exact-hash check. These are material limits on the interpretation of the scores, not details remedied by setting a seed.

5. Shared preprocessing

All inputs are resized to 128×128 pixels using nearest-neighbour interpolation. Colour frames are converted from OpenCV BGR to grayscale, and Otsu thresholding produces a binary image. Pixel values are divided by 255 and a single channel is retained. The same function is used by training, evaluation, the CLI and the desktop interface. There is no separate preprocessing path that could silently change the input scale at deployment.

The surviving training script specifies 128-pixel inputs despite the historical paper's discussion of 64-pixel images. This revision follows the code for the CNN re-run and states the difference. The static classifier requires a suitable hand crop; automatic hand localisation is outside this image-classification pipeline.

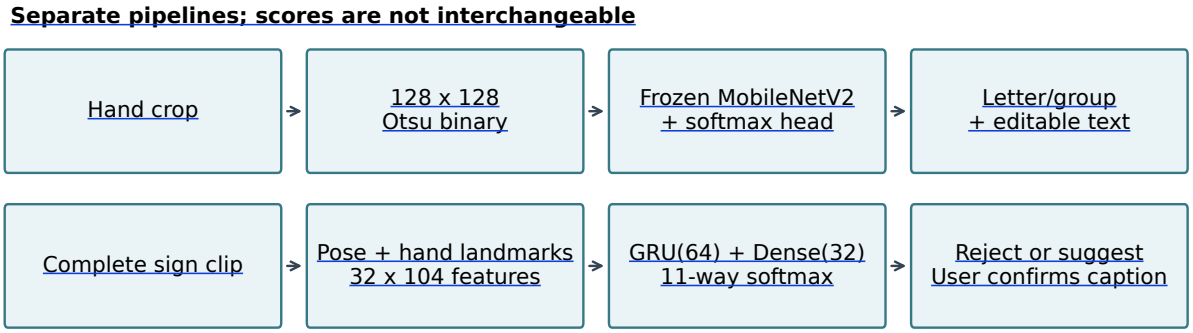


Figure 1: Inference architecture. Static classification and complete-clip sentence recognition use separate feature contracts and label spaces. The original CNN comparison is specified in the method text. Manual confirmation is an interface safeguard, not evidence of recognition accuracy.

6. Models and training setup

The reconstructed CNN uses two valid 5×5 convolutions with 32 filters, a 5×5 max-pooling layer and dropout 0.25. Two valid 3×3 convolutions with 64 filters follow, then 2×2 max pooling and dropout 0.25. Flattened features feed a 512-unit ReLU dense layer, dropout 0.5 and a softmax classifier. All convolutional layers use ReLU. The historical experiment has 36 outputs and preserves the architecture recorded in `ishara.py` at the thesis tag.

Adam uses its default learning rate of 0.001. Sparse categorical cross-entropy is minimised for 12 full epochs with batches of 18. Historical augmentation retains the surviving script's shear 0.2, zoom 0.2 and horizontal flips. These settings are preserved for traceability rather than endorsed as linguistically valid: a reflected sign can be inappropriate, and augmentation choices require review by sign-language users. Unlike the old fixed generator step counts, the modern loop visits

each training batch in an epoch.

The comparison repeats the binary input across three channels and scales it to $[-1, 1]$ for ImageNet MobileNetV2. Its backbone is frozen. Global average pooling produces 1,280-dimensional features, followed by dropout 0.2 and a new softmax head. Frozen features are cached once on CPU. The head is trained for 12 epochs using Adam, batch size 18 and no geometric augmentation. The exported end-to-end model still accepts image tensors, not cached vectors.

This is a small transfer baseline, not an architecture-only ablation: pretraining and augmentation differ. Both historical models nevertheless use precisely the same training, validation and test examples, making the difference in their measured outcomes inspectable. The public-data run trains another head in its distinct grouped class space.

Python, TensorFlow, Keras, OpenCV and other direct dependencies are pinned. TensorFlow random seeds and deterministic operations are enabled, and CPU execution is used. Environment metadata and model hashes accompany every experiment. A fixed seed improves repeatability within an environment; it does not guarantee bit-identical floating-point results across all processors and library versions.

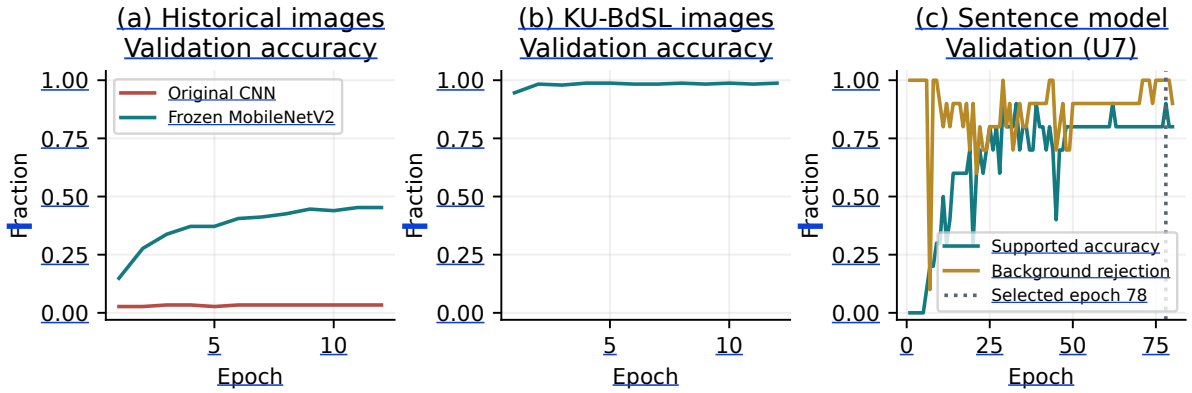


Figure 2: Unsmoothed validation histories from the saved runs. Panels (a) and (b) show all 12 image-training epochs. Panel (c) shows all 80 temporal epochs and the checkpoint selected using U7 validation. Test clips are not plotted or used for checkpoint selection. Training accuracy is omitted because the historical logger records the last batch rather than an epoch aggregate.

7. Metrics

Test predictions are obtained once by argmax of each output distribution. Accuracy is the number correct divided by test support. Per-class precision, recall and F1 are reported with support; undefined values are set to zero. The confusion matrices contain raw counts and use the saved class order. Softmax outputs are presented as uncalibrated scores, never as a validated probability of correctness.

8. Static classification results

The historical comparison measures two models on the same image-level split. The public-data row uses a different collection and grouped label space and should be interpreted separately. These are additional experimental measurements, reported independently of the original

Table 1: Static classification on fixed image-level test partitions. Historical models share one split; KU-BdSL is a separate task.

Experiment	Classes	Train	Val.	Test	Acc. (%)	Macro F1
Historical CNN	36	709	148	148	3.38	0.0018
Historical MobileNetV2	36	709	148	148	52.70	0.5072
KU-BdSL MobileNetV2	30	1020	240	240	99.58	0.9958

preprint's accuracy statements. One seed and one run per configuration do not establish robustness or a confidence interval over training variability.

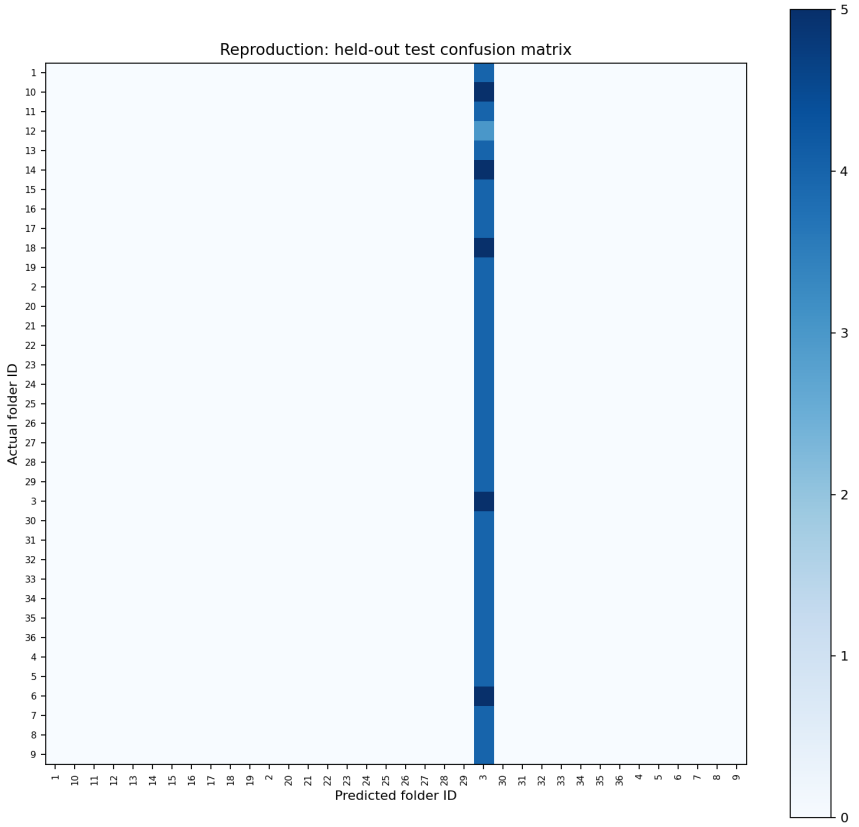


Figure 3: Historical CNN confusion matrix for 148 test images. Rows denote actual folder IDs and columns denote predicted IDs; entries are raw counts.

Figure 3 shows held-out predictions; the raw counts are available as CSV. Concentration in a small number of prediction columns indicates poor discrimination across the historical classes.

The public-data and historical tasks differ in image quality, class definitions, collection conditions and support per class. Cross-dataset score differences cannot isolate the effects of better software, representation learning or sample size. No comparison with a literature accuracy claim is used as evidence of improvement.

9. Per-class metrics

The following are the complete per-class precision (P), recall (R), F1 and support (N), rounded only for display. Unrounded values are in the committed CSV and JSON reports. The numeric IDs preserve the original folder labels; hyphenated public IDs denote grouped characters. Class order in the model and plots is lexicographic and stored explicitly.

Historical CNN

ID	P	R	F1	N
1	0.000	0.000	0.000	4
10	0.000	0.000	0.000	5
11	0.000	0.000	0.000	4
12	0.000	0.000	0.000	3
13	0.000	0.000	0.000	4
14	0.000	0.000	0.000	5
15	0.000	0.000	0.000	4
16	0.000	0.000	0.000	4
17	0.000	0.000	0.000	4
18	0.000	0.000	0.000	5
19	0.000	0.000	0.000	4
2	0.000	0.000	0.000	4
20	0.000	0.000	0.000	4
21	0.000	0.000	0.000	4
22	0.000	0.000	0.000	4
23	0.000	0.000	0.000	4
24	0.000	0.000	0.000	4
25	0.000	0.000	0.000	4
26	0.000	0.000	0.000	4
27	0.000	0.000	0.000	4
28	0.000	0.000	0.000	4
29	0.000	0.000	0.000	4
3	0.034	1.000	0.065	5
30	0.000	0.000	0.000	4
31	0.000	0.000	0.000	4
32	0.000	0.000	0.000	4
33	0.000	0.000	0.000	4
34	0.000	0.000	0.000	4
35	0.000	0.000	0.000	4
36	0.000	0.000	0.000	4
4	0.000	0.000	0.000	4
5	0.000	0.000	0.000	4
6	0.000	0.000	0.000	5
7	0.000	0.000	0.000	4
8	0.000	0.000	0.000	4
9	0.000	0.000	0.000	4

Public-data MobileNetV2

ID	P	R	F1	N
2433	1.000	1.000	1.000	8
2434	1.000	1.000	1.000	8
2435	1.000	1.000	1.000	8
2453	1.000	1.000	1.000	8
2454	1.000	1.000	1.000	8
2455	1.000	1.000	1.000	8
2456	1.000	1.000	1.000	8
2457	1.000	1.000	1.000	8
2458	1.000	1.000	1.000	8
2459	1.000	1.000	1.000	8
2460-2479	1.000	1.000	1.000	8
2461	1.000	1.000	1.000	8
2462	1.000	0.875	0.933	8
2463	1.000	1.000	1.000	8
2464	1.000	1.000	1.000	8
2465	1.000	1.000	1.000	8
2466	1.000	1.000	1.000	8
2467-2472	1.000	1.000	1.000	8
2468-2510	1.000	1.000	1.000	8
2469	1.000	1.000	1.000	8
2470	1.000	1.000	1.000	8
2471	1.000	1.000	1.000	8
2474	1.000	1.000	1.000	8
2475	1.000	1.000	1.000	8
2476-2477	1.000	1.000	1.000	8
2478	1.000	1.000	1.000	8
2480-2524-2525	1.000	1.000	1.000	8
2482	1.000	1.000	1.000	8
2486-2488-2487	0.889	1.000	0.941	8
2489	1.000	1.000	1.000	8

The transfer comparison's complete 36-class table is distributed as `reports/mobilenet/per_class.csv`. The public matrix appears below; its axis labels are numeric Unicode/group IDs rather than inferred letter names.

10. Error analysis and qualitative evidence

The examples above are the first correct and first incorrect public test predictions in the saved manifest, when each exists. Their source paths and full scores are stored in `examples.json`. Both originate from held-out images; they are illustrations rather than a random sample or an estimate of error frequency. Dataset credit: Jim, Rafi, Akon, Biswas and Nahid, KU-BdSL v4, CC BY 4.0. Images are resized derivatives.

Errors can be inspected at class level through recall and at prediction level through the complete results page. Low recall means many members of a class were missed; low precision means

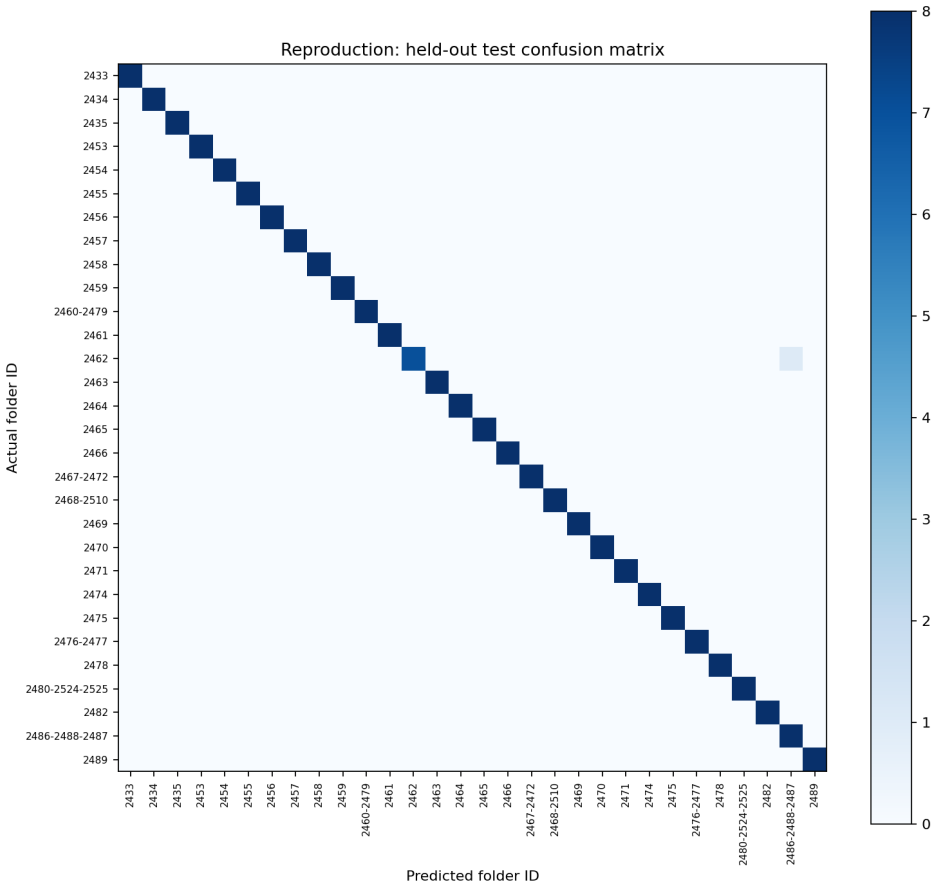


Figure 4: KU-BdSL MobileNetV2 confusion matrix for 240 test images in 30 grouped classes. Rows are actual IDs and columns predicted IDs.



Actual ID 2433; predicted 2433. Score 100.0%.



Actual ID 2462; predicted 2486-2488-2487. Score 25.5%.

Figure 5: A correct and an incorrect held-out KU-BdSL prediction. Scores are uncalibrated softmax outputs. Images: Jim et al., KU-BdSL v4, CC BY 4.0; resized derivatives.

predictions assigned to that class frequently belonged elsewhere. The released audit shows every test image and score, including failures, so readers can investigate competing hand configurations and background effects without relying on a few chosen successful screenshots.

The evidence does not establish a causal explanation for any individual error. Similar hand-shapes, threshold sensitivity, cropping and limited samples are plausible hypotheses for further investigation, not experimentally isolated causes. The binary preprocessing discards colour and texture; this may remove useful information and may interact poorly with features pretrained on natural colour images. An RGB baseline would be a useful future experiment, but it was not measured in this revision.

11. Implementation and software validation

The Python implementation provides command-line and PyQt5 interfaces for images, recorded video and cameras. Automated tests cover Otsu preprocessing, tensor contracts, image decoding, Keras prediction and video inference. Synthetic image and video fixtures test software behaviour independently of sign-recognition accuracy. Recorded-file sampling follows media timestamps, and tracking state resets between inputs.

The desktop video-input flow was also tested with a supported clip, a background false accept and a cropped BRAC tutorial greeting. The supported clip was recognised correctly; both unsupported inputs produced incorrect supported captions, including a coffee-related sentence for the greeting. Suggestions required confirmation before transcript insertion. No weights or thresholds were adjusted after these checks. The first two clips were previously evaluated examples, so this test measures integration consistency rather than new recognition accuracy. A separate letter-composition demonstration uses selected correctly classified held-out images; it is not continuous signing.

12. Moving-phrase feasibility experiment

The supplied Grameenphone and BRAC tutorials demonstrate moving words and sentences, which differ from isolated alphabet signs. A separate CPU prototype extracts upper-body and hand landmarks from signer crops, normalises coordinates using the shoulders, and compares resampled sequences with dynamic time warping. Printed caption pixels are not model features; they are used only to annotate reference and test intervals manually. Ten phrases have one reference example each from the first tutorial. The second tutorial contains a different signer and supplies four supported and eight unsupported test segments.

With a distance threshold of 0.20 and a relative margin of 0.10 specified before evaluation, the prototype recognises 1 of four supported phrases and rejects 4 of eight unsupported segments. These are tiny, manually segmented feasibility measurements, not continuous-transcription accuracy or a public-use validation. Its streaming preview and unknown state are software functions; they do not remedy the poor observed cross-signer result. More labelled examples, separate validation signers, representative unknown gestures and sign-language expert review are required before deployment.

Segment annotations and temporal errors are reported separately from the static image metrics.

12.1. Bounded sentence model

A follow-up experiment uses 100 selected clips from the public Ban-Sign-Sent-9K-V1 repository (`https://huggingface.co/datasets/banglagov/Ban-Sign-Sent-9K-V1`). Ten text-diverse target sentences and ten background sentence types were selected before recognition outcomes. Encoded signer IDs U4/U5/U6 supply 60 training clips; U7 supplies 20 validation clips; U19 supplies 20 test clips. There is one clip per sentence and ID. Identity and consent were not independently verified, and the inspected dataset card has no explicit licence; source footage is not redistributed.

An 11-class GRU classifier (64 units, dropout 0.3, dense 32, softmax including background) uses 32-step motion/shape landmark sequences, Adam 0.001, batch 8, 80 epochs, temporal warp/noise augmentation and seed 2026. Validation selects the checkpoint and rejection threshold, with DTW as an alternative backend. The selected GRU obtains 6/10 supported captions correct and rejects 8/10 background clips on U19; 6 of 10 accepted captions are correct. The

two false accepts illustrate the danger of confident captions. Background sentence types occur in training, so rejection does not establish novel-utterance detection. An earlier conservative baseline rejected all clips; its U19 outcome was already observed before this follow-up, which is exploratory rather than a wholly blind confirmation. Full clip-level predictions, class metrics and confusion counts accompany the code. Automatic sentence boundaries and general conversation translation remain unevaluated.

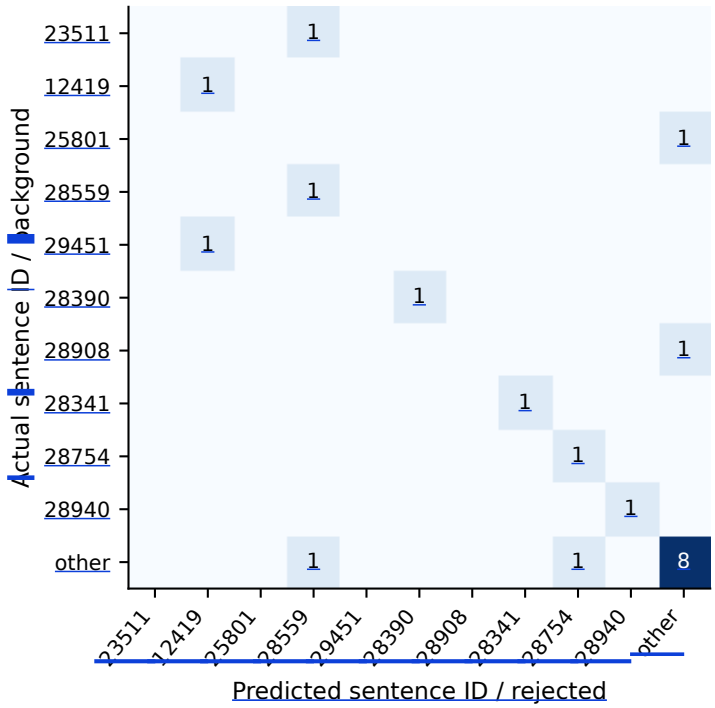


Figure 6: Raw confusion counts for 20 U19 sentence-test clips: one per supported sentence and ten background clips. Numeric IDs correspond to the published vocabulary manifest; "other" pools unsupported sentence types and rejected outputs. Six supported captions are correct, and eight background clips are rejected; two background clips are falsely captioned. Background types also occur in training, and a prior baseline outcome on this signer was observed.

13. Limitations, ethics and accessibility

Deployment evaluation requires Deaf-user-led task design, resolved data permissions and consent, independent signers and sessions, novel unsupported utterances, continuous-signing error analysis and measured end-to-end latency. These remain open research and validation requirements.

Small test supports make class metrics unstable. Neither static-image dataset permits a verified signer-disjoint partition in the layout used here, and capture-condition dependence can inflate image-level results. Historical annotations and their display mapping require expert review. The static classifier has no explicit unknown-sign, no-hand or out-of-distribution rejection class; the separate temporal background class does not solve arbitrary unknown recognition. A high softmax score can still be wrong.

KU-BdSL version 4 provides a documented CC BY 4.0 licence and participant-consent statement. That documentation does not transfer to the historical thesis images, whose consent records are absent. The software's MIT licence does not relicense third-party datasets. No unli-

censed face video is incorporated. Historical paper text and figures remain under their original licence, and unchanged v1 is supplied for comparison.

An accessibility claim would require evaluation with Bangla Sign Language users, including variation in signing practice, mobility, lighting, skin tone and camera access. This project has not performed such a study. Static alphabet classification is a constrained research exercise and should not be presented as a replacement for interpreters or a complete communication service.

14. Code and data availability

Code, pinned requirements, manifests, predictions and tests are available at `https://github.com/iamnajib71/Real-Time-Bangla-Sign-Language-Detector`. Versioned models, matching label maps and checksums are provided with release `v2.0.1`: `https://github.com/iamnajib71/Real-Time-Bangla-Sign-Language-Detector/releases/tag/v2.0.1`. The public-data downloader retrieves KU-BdSL v4 directly from its publisher and verifies each supplied SHA256. KU-BdSL v4 is available from its publisher [3] under CC BY 4.0. The historical image records and split manifests are preserved in the repository, although original consent documentation is unavailable. Ban-Sign-Sent source clips remain with the original provider; their reuse permissions are unresolved, and those clips are not redistributed.

15. Changes from version 1

The original preprint is unchanged. This version specifies the executable CNN and its differences from the earlier method description; adds audited partitions, model-release links, per-class metrics and error analysis; and reports separate public-data and bounded sentence-recognition experiments. Original accuracy statements are not treated as results of these experiments. Sparse-autoencoder, depth and MATLAB procedures described previously are not reproduced because corresponding executable evidence is unavailable.

References

[1] N. Hassan. Bangla Sign Language Gesture Recognition System: Using CNN Model. ScienceOpen Preprints, 2022. Non-peer-reviewed. DOI: 10.14293/S2199-1006.1.SOR-.PPUF56Q.v1.

[2] A. A. J. Jim, I. Rafi, M. Z. Akon, U. Biswas and A.-A. Nahid. KU-BdSL: An open dataset for Bengali sign language recognition. Data in Brief 51, 109797, 2023. DOI: 10.1016/j.dib.2023.109797.

[3] A. A. J. Jim et al. KU-BdSL: Khulna University Bengali Sign Language dataset. Mendeley Data, version 4, 2023. DOI: 10.17632/scpvm2nbkm.4.

[4] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L.-C. Chen. MobileNetV2: Inverted Residuals and Linear Bottlenecks. 2018. DOI: 10.48550/arXiv.1801.04381.



Article title: Bangla Sign Language Gesture Recognition System: Using CNN Model

Authors: Nazmul Hassan[1]

Affiliations: Department of Electrical and Computer Engineering, North South University[1]

Orcid ids: 0000-0001-6116-017X[1]

Contact e-mail: iam.najib16@gmail.com

License information: This work has been published open access under Creative Commons Attribution License http://creativecommons.org/licenses/by/4.0/, which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited. Conditions, terms of use and publishing policy can be found at https://www.scienceopen.com/.

Preprint statement: This article is a preprint and has not been peer-reviewed, under consideration and submitted to ScienceOpen Preprints for open peer review.

DOI: 10.14293/S2199-1006.1.SOR-.PPUF56Q.v1

Preprint first posted online: 25 October 2022

Keywords: BdSLtraining data, CNN, structure-building, Deep Learning, Recognition

Bangla Sign Language Gesture Recognition System Using CNN Model

Nazmul Hassan Najib, Mushfiqur Rahman Dipto and Jubaer Bin Faruk

Department of Electrical and Computer Engineering
North South University
Dhaka, Bangladesh

iam.najib16@gmail.com

***Abstract* - Sign language is a form of communication used by the deaf and dumb. However, integrating them into society is challenging because most people do not speak their language. It is a method of generating and disseminating ideas, information, points of view, facts, and sentiments to establish a shared understanding. Unfortunately, almost 5% of the world's population is not blessed with the ability of verbal communication. The situation is the same for Bangladesh. This paper presents a simple, low-cost Bangla sign language (BdSL) recognition model with maximum accuracy to convert signs into Bangla text. We've manually captured images for Bangla sign language following the BdSL model and implemented the dataset to train the system and show output. We used neural networks as a deep learning method to train individual signs to achieve our goals in the suggested model. Using a CNN, we separated the dataset into two parts: training data and test data. The action is mapped to the relevant text in the training data using image processing techniques, and neural networks, so raw images or videos are turned into text that can be read and comprehended with accuracy near 92%. Finally, the system will turn the user's signs and motions into text. It is referred to as "sign language recognition." In the future, This research can also open doors to numerous other applications, like sign language tutorials or dictionaries, and can also help the deaf and dumb search the web or send emails more conveniently. We hope this system will be an excellent opportunity for people who have speaking or hearing disabilities by helping them explore more.**

Keywords - BdSL; CNN; structure-building; training data; Deep Learning; Recognition

I. Introduction

Sign language is the only way for deaf and hard-of-hearing persons to communicate and express themselves [1]. Hard hearing is a loss of aptitude that impairs their ability to tune in and castrates their hearing, and dumb refers to a lack of skill that impairs their ability to communicate and renders them speechless [2]. 2.4 million People utilize sign language in Bangladesh, yet most do not understand it [3]. Bangla is the mother tongue of both the general public and the disabled. Hence there should be a universal medium for everyone. Disabled persons fall behind the general public due to communication issues [4]. This problem can be solved by using a versatile medium.

On the other hand, the United States has been using its own created sign language pattern known as American Sign Language (ASL) [5]. We have to reduce the communication gap so that hard of hearing people can express themselves more easily. Only a few people have worked with the Bangla sign language concept. Deaf and mute persons find it challenging to learn their signs because no relevant model can be used as a communication strategy. We are motivated to propose the system as it is critical to develop a system that converts sign language to text, allowing deaf people to communicate with the general public.

II. Literature Review

Language developments and an emerging body of research in education have validated the NAD's historical views on ASL and its value in educating deaf and hard-of-hearing children. Children's natural ability to learn ASL should be recognized and used to enhance their cognitive, academic, social, and emotional development. As a result, the NAD advocates for a bilingual approach for deaf and hard-of-hearing children. Deaf and hard-of-hearing children must have the right to early and comprehensive exposure to ASL as a primary language alongside English.

According to the National Institute on Deafness and Other Communication Disorders (NIDCD), the 200-year-old American Sign Language is a complete, complex language (letter gestures are only a tiny part). Still, it is the primary language of many deaf North Americans. As a result, developing a system that recognizes sign language will enable the deaf and hard-of-hearing to communicate more effectively using modern technologies. This article will look at various CNN architectures and how they perform when classifying Sign Language [6].

People with hearing and vision loss are excluded from participation in their family environment and society, their rights are ignored, and their needs are not met. This rejection can lead to a subhuman existence if it is not recognized and understood. It prompted CDD to intervene to raise awareness and understanding of deafblindness and provide support to people and families affected to uphold fundamental rights and give a voice to the most marginalized disabled people[7]. Previous research in referential communication has found that typically developing children progress in communicating a specific referent to a listener. In one paradigm, subjects were given lists of word pairs, one of which was designated as the referent. It was demonstrated that communicating about

referents found in word pairs that were somehow associated was more difficult than speaking about referents found in different word pairs. The current study extended this methodology to children with learning disabilities. There were few differences in the groups' performance on these referential communication tasks. The importance of vocabulary and concept development in referential communication is one of the implications [8]. A further analysis limited to the deaf group revealed that these results were not affected by the age of subjects' acquisition of ASL. This research adds to our understanding of the processes by which the handshape component of sign forms is recognized in a sign language, the role of language experience, and the extent to which these processes can be explicitly considered linguistic [9]. This paper investigates the relationship between movement, motion, and thought in humans and nonhuman primates and its role in language evolution. Human gestures (mainly symbolic gestures) may provide a specific link between action and mental representation. It is kinesthetically close to the action while also being symbolic. Nonhuman primates communicate frequently and in a variety of ways.

On the other hand, their gestures resemble incomplete movements and lack the symbolic elements that characterize many human gestures. We contend that gesture played an essential role in human evolution's transition to symbolic thought and language, following a cognitive leap that allowed the motion to incorporate representational elements [10]. Hearing loss is defined as diminished hearing caused by a variety of factors. It can be inherited or acquired later in life. Mild hearing loss to profound hearing loss is all possibilities [11].

The following paper presents an approach for recognizing static fingerspelling in Japanese Sign Language on RGB images. Two 3D articulated hand models were created to generate synthetic fingerspellings and supplement a simple hand gestures dataset. The first method used advanced graphics techniques to rasterize photorealistic gestures using a skinned hand model. A modified Generative Adversarial Network was used to post-process gestures rendered using more straightforward lighting techniques in the second approach. A hand segmentation term has been added to the GAN's loss function to avoid the generation of unrealistic fingerspellings. The proposed ResNet34-based segmentation network was used to segment the hand in images with complex backgrounds. An ensemble of fine-tuned and trained-from-scratch neural networks recognized the fingerspelled signs. The experimental results show that sufficient training data can achieve a high recognition rate on RGB images. The JSL dataset with pixel-level hand segmentations can be downloaded [13].

Hearing and speaking impairments are extremely debilitating handicaps for humans. People with such impairments frequently communicate using sign language, which is very different from spoken and written language, often leading to communication difficulties when a translation medium is not readily available. This causes a significant problem in the affected people's daily lives, particularly in Bangladesh. A simple, accurate, and portable translation medium that allows two-way communication between an average person and an impaired person is required to address this issue. Speech recognition and conversion to sign language is a critical component of such a medium, especially since Bangla is a complex language to work with. This paper describes the development of a smartphone application that provides a two-way communication platform using a speech-to-sign language system and a text-to-speech system. The application's testing yielded satisfactory results, with an average accuracy of 84.71 percent in various test environments [14].

This paper proposes an Artificial Neural Network-based hand sign language recognition framework for various Bangla alphabets (ANN). The boundary edge of the hand sign area is extracted using the canny edge detector, and the hand sign region of interest is extracted (ROI). Finally, features are removed from the hand sign ROI (FCC) using Freeman Chain Code. The ANN is used for training and categorizing the images of hand signs. The proposed method is tested with various hand sign images, and the results are presented to show its efficiency and effectiveness [15]. A real-time computer vision-based Bengali Sign Language (BdSL) recognition system is presented in this paper. The system recognizes the likely hand in the captured image. The system employs Haar-like feature-based cascaded classifiers to detect the writing in each frame. The system extracts the hand sign from the detected hand area based on the Hue and Saturation value corresponding to human skin color. The system converts the hand sign to the binary image after normalization. The binary images are then classified using the K-Nearest Neighbors (KNN) Classifier by comparing them to pre-trained binary images of hand signs. The system can recognize six Bengali vowels and thirty Bengali consonants. The system is trained using 3600 (3610) training images. Each of the ten signers performed ten signs for each corresponding Bengali alphabet. It is tested using another 3600 (3610) pictures of ten signers. The system achieved 98.17 percent vowel recognition accuracy and 94.75 percent recognition accuracy for consonants [16].

Researchers are having a difficult time collecting hand gesture data for sign language. This article examines Ishara-Lipi, the first complete isolated characters dataset of Bangla Sign Language (BdSL). It will increase interaction between the hearing-impaired community and the general public. The dataset contains 50 sets of 36 Bangla primary sign characters collected with the assistance of various deaf and available volunteers. Bangla Sign Language sign characters have six vowels and thirty consonants, allowing them to fingerspell all Bangla words. After removing errors and pre-processing, 1800 character images of Bangla Sign Language were included in the final state of the Ishara-Lipi dataset. This dataset could be used to create a computer vision-based system or any other type of system that allows users to look up the meaning of a BdSL sign [17].

We will now create a more assertive approach to image classification, which will naturally extend to entire Neural Networks and Convolutional Neural Networks. The method will consist of two major components: a score function that converts raw data into class scores and a loss function that quantifies the agreement between predicted scores and ground truth labels. The problem will then be recast as an optimization problem in which we will minimize the loss function concerning the score function's parameters [18].

III. Methodology

Because the computer system cannot recognize the sign, any type of sign language identification is extremely difficult. The researchers put in a lot of effort and used various methods to identify the signal using a computer system.

Our proposed system obtained an open-source dataset from the BdSL recognition model that is open to the public. 'Ishara-Bachon' and 'Ishara-Lipi' are the digits and character dataset names, respectively [19]. We initially looked at some previous work related to our research, such as segmentation-robust modeling for sign language recognition [20] and sign language and human activity recognition [21]. Still, we primarily used our technique for Bangla sign language recognition. Based on the Microsoft Research group's publication on visual document analysis [22], we studied the application of convolutional neural networks for feature learning. However, we believed that a more uncomplicated strategy would still produce competitive accuracy results.

A. Block Diagram

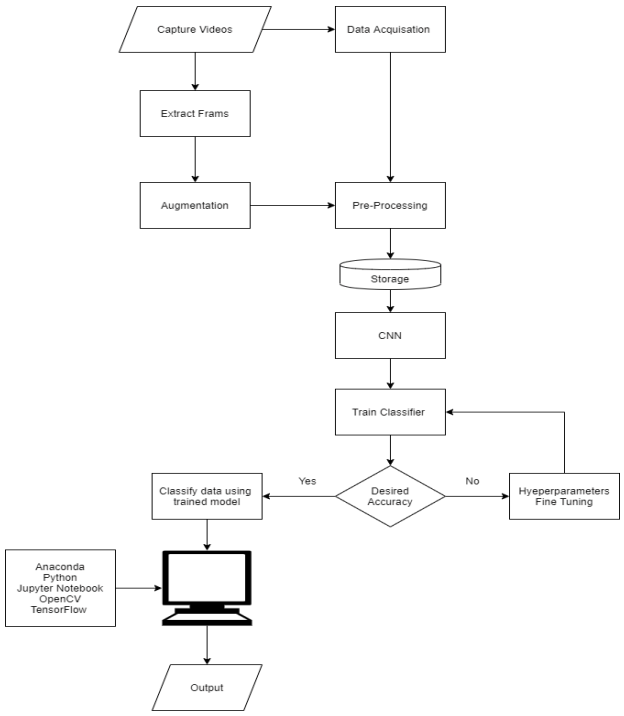


Fig. 1. Data Flow Diagram

The overview of the proposed system is illustrated below with data acquisition and pre-processing. The video frames will be submitted as the input to the system, and the output will be the predicted sign digit presented in the Bangla language. Video frames as pictures will be augmented to change the dataset, and the model was trained using CNN. The model will be stored and loaded with OpenCV to recognize the signs in real-time.

B. Use Case Diagram

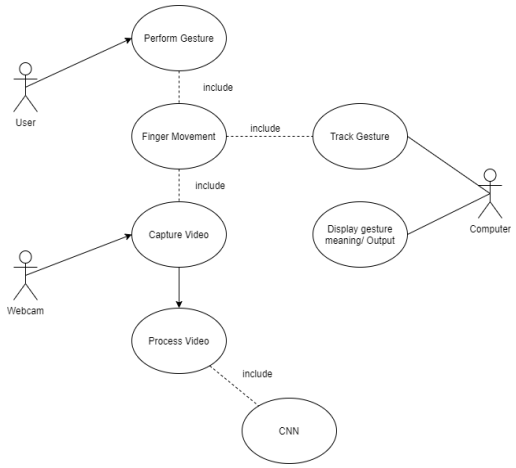


Fig. 2. Case Diagram

Use case diagrams are also known as behavior diagrams. They depict behaviors that specific systems (subjects) should or may do in partnership with the platform's multiple stakeholders (actors). To the actors or other users of the system, each user should deliver some measurable and valued output. So the primary workflow of our system will be the followings:

1) Capture Video: The primary purpose of recording a video will be to process it. The webcam is the main character, while the user is the supporting character. The user should have or have access to a webcam. The live stream will be collected here, and operations will be carried out. If the program fails, there'll be an exception.

2) Process Video: The purpose here is to process the videos that have been received. Webcam is the main character. A webcam is used to capture live video, transformed into photographs. Various operations are carried out to improve related properties and gesture actions.

3) Display Gesture Meaning: The output/meaning based on tracking and gesture recognition will be shown here. The computer is the main character, with the user and camera as minor characters. The user must have or have access to a camera as a precondition. The network will be used to recognize the improved photos.

C. The optimization algorithm

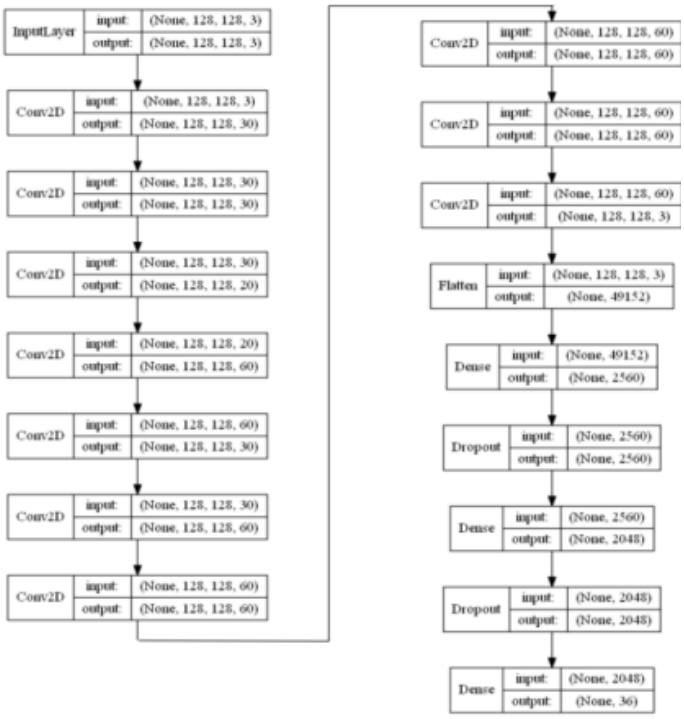


Fig. 3. The neural network architecture is depicted in this diagram.

The optimization algorithm chosen can significantly impact the outcome in Deep Learning. Computer vision and learning go hand in hand. According to a recent study, the cross-entropy function outperforms classification and mean square errors. The weight changes don't get smaller and smaller due to cross-entropy error, so training isn't likely to stall. Using an automated Learning Rate reduction strategy, make the optimizer converge faster and closer to the global minimum of the loss function. The learning rate is the number of steps taken to walk through the least amount of loss. If a faster learning rate is used, the system will soon converge and become locked in a local minimum rather than a global minimum. After each epoch, the model dynamically decreases the learning rate by monitoring validation accuracy to maintain the benefit of quick computation time with a high Learning Rate.

IV. Implementation

A. Dataset Description

Had the data for training and testing came from a dataset we created by ourselves. A digital camera captured 1005 samples of the 36 signed Bangla letters and the accompanying depth data. A person signs the corresponding letter directly facing the Digital camera in each piece. The following is a representation of some of the dataset's unprocessed raw images:

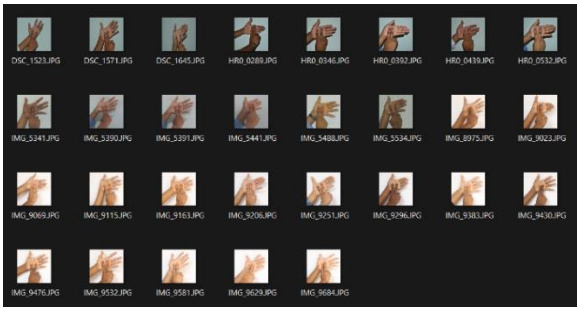


Fig. 4. Example Images in Dataset

B. Technical Approach

The PILLOW library, an image processing toolkit, and the sklearn, the decomposition package, valuable for its matrix optimization and decomposition functionality, were used to pre-process the data. The fundamental objective has been broken down into four stages- Image Capturing, Data Labeling, Image Cropping, Resizing, and Converting to Gray-scale. We started by resizing the image to 64*64 pixels, then converting RGB to grayscale.

a) Capturing Data: Open-source data is being gathered, and photographs of bare hands are being taken. The white background is chosen as the background color, and we then take one shot at a time. Because of the high resolution, a DSLR camera is used to photograph hand signs. We aim to reduce white space from the hand's edge while collecting photographs.

b) Labelling Data: Labeling data is critical for reducing noise in capturing and improving image quality. We'll write our script after gathering data. Some BdSL signs, for example, are very similar to one another. Images of this type of sign look familiar and add to the sense of experimentation. We'll do it by categorizing the signs individually. Different classes/folders exist for different signs. For digits, ten different directories (numbered 0, 1, 2, 3, 4, 5, 6, 7, 8, 9) were created. On the other hand, Bangla has roughly 49 characters, but not in sign language. There are only 36 characters in Bangla Sign Language. As a result, we have 36 characters after naming them with a numeric convention ranging from 1 to 36. (1, 2, 3...9). After that, we organize the photographs by categorizing them into distinct classes. The naming scheme for all characters is shown below.

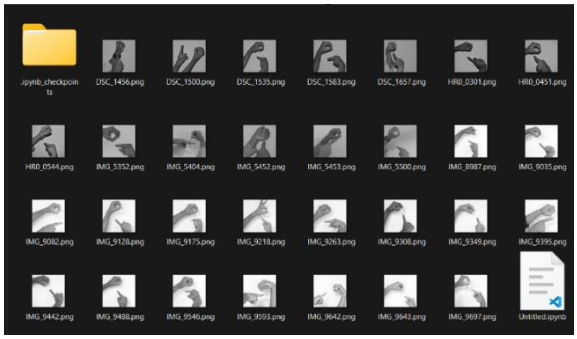


Fig. 5. Image for Segmentation

c) Cropping Data: The captured image cannot be used without cropping for sign recognition purposes. Cropping is required to obtain an idea that may be used for further experimentation. We're manually cutting photos to remove noise and keeping track of the height and width ratios for further processing. The hand region is cropped from the images. Cropping is used to reduce as many white areas as feasible.

d) Resizing Data: All photos will be scaled in a standard format using a python cv2 script to make our data more useful. We're writing a script that will go into each folder containing images and do the identical operation on each image. The image will be resized to 64 * 64 pixels by the scripts.

e) Converting Data to Grey-Scale: The script will convert RGB to grayscale after scaling. All photos will be prepared in a uniform format, with the same pixels in height and width. All sign photos will be reduced to 64 * 64 pixels and resized. There will be 36 characters, with six Bangla vowels and thirty consonants. All images will be saved in the.jpg data type. The photos will be collected, resized, and transformed from RGB to grayscale.

C. Application and Performance

We have tried to implement some popular methods for quick image segmentation, particularly hand segmentation. Using a Canny edge detector to discover relevant "things" in the camera's field of vision was one of the ways we investigated. To make a solid, continuous mask, the edges were dilated, and then all remaining holes in the mask were filled.

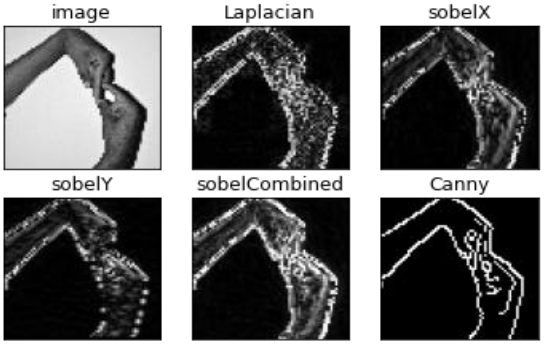


Fig. 6. cannyEdgeDetector Implementation

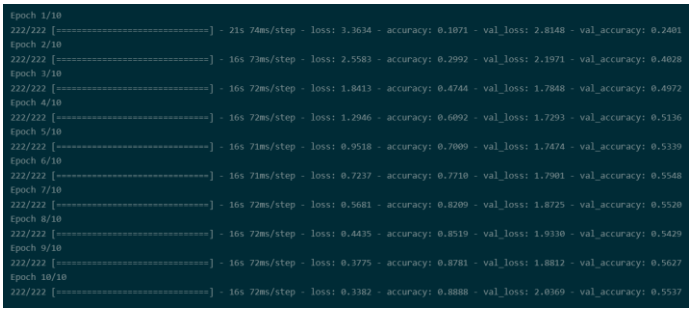


Fig. 7. Value Accuracy from training

After that, just the significant portions were photographed to eliminate all background clutter objects. This method is based on the notion that the most oversized items in segmentation are also the ones of most significant interest. We used solely color information to test the two algorithms for skin segmentation. The initial method involves fitting a 2D Gaussian curve to the skin color and then estimating the likelihood of a given color pixel being skin using this fitted Gaussian. First, we gathered skin patches from 40 different photographs found on the internet. Each skin patch was a rectangular skin region that was contiguous. Skin patches were obtained from people of various races for our model to accurately forecast skin regions for a wide range of skin colors. After that, the colors were adjusted as follows:

$$r = \frac{R}{R+G+B}, b = \frac{B}{R+G+B}$$

It is ignored because the g component is linearly dependent on the other two. The 2D Gaussian mean and covariance matrix (with r and b as the axes) is calculated as follows:

$$\text{Mean } m = E[x], \text{where } x = [r, b]^T$$
$$\text{Covariance } C = E[(x-m)(x-m)^T]$$

The likelihood of skin for any pixel in a given test image can be calculated using this Gaussian fitted skin color model. If the chromatic pair value of a pixel is (r, b), the likelihood of skin for that pixel is:

$$Likelihood = e^{[-0.5(x-m)^T C^{-1}(x-m)]}, where\ x = [r, b]^T$$

Finally, we used a threshold to determine if it was skin or non-skin. However, this method did not produce consistently good results and failed to detect weakly light skin areas. These alarming results didn't fulfill the hand segmentation quality criteria we needed to acquire consistent feature extraction from the model's learning layer

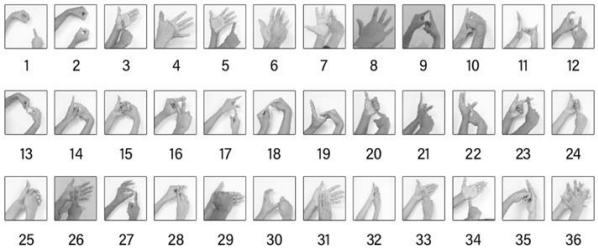


Fig. 8. Visualization of processed Database Images

We filtered out the hand region and deleted the face/other background pixels that would have been discovered using the acquired depth data after detecting the skin regions with great accuracy. The hand was presumed to be the focus of interest in each given frame and thus the closest skin-colored item in the camera's view. Each image was cropped and reduced to fit into a 64x64 pixel square. We then used this segmentation model to build a supplementary dataset containing hand gestures for each BDSL alphabet's 36 letters.

D. Classification and Feature Learning

To accomplish the actual recognition training phase of the project, the extracted data from hand pictures were loaded into an autoencoder. An unsupervised learning algorithm is implemented at this stage. All of the data samples are fed into the sparse autoencoder. The segmentation block's input data consists of 64x64 pixel pictures. Initially, a sparse autoencoder with a 64x64 node input layer and a 100 node hidden layer is chosen. The cost function was optimized using L-BFGS. This was running for around 400 iterations to get estimates of the weights. The autoencoder now knows a collection of features that are similar to edges.

The Softmax classifier uses the cross-entropy loss. The Softmax classifier gets its name from the softmax function, which squashes the raw class scores into normalized positive values that sum to one so that the cross-entropy loss can be applied [23].

$$L_i = -f_{y_i} + log \sum_j e^{f_j}$$

The next step is to classify the thirty-six letters using the autoencoder training's features. The output of the autoencoder's hidden layer is sent into a softmax classifier, which divides the data into ten groups. The L-BFGS optimization function is used to train the softmax classifier once more. After around 40 iterations, this algorithm converges. We used the remaining 30 images per letter (for 1005 images) as our test set to assess the system's accuracy. The overall system can be seen below:

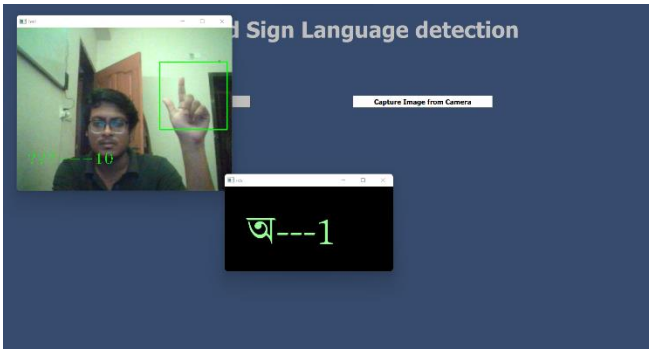


Fig. 9. Overall Preview of our proposed system

V. Experimental Results and Discussion

We went over the details of our implementation of the hand segmentation, unsupervised feature learning, and classification sub-blocks in the preceding sections. This section presents our system's performance in tables and graphs. The accuracy of our classifications is our primary evaluation metric. We want to evaluate how accurate our model is at classifying and predicting the signed letter in an unsegmented image of a person signing a letter. It is hoped to achieve a classification accuracy of roughly 98 percent, similar to many sign language identification scores. We used a learning curve to represent the training error and test error as a function of the Size of the training set as a preliminary diagnosis. The learning curve we acquired is depicted in the graph below, where the X-axis represents the error, and Y-axis represents the Size of the data set:

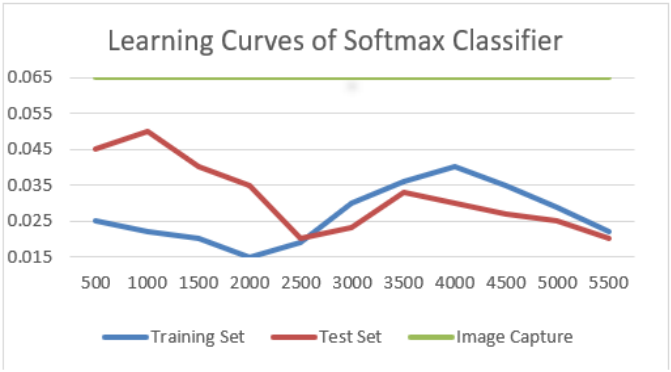


Fig. 10. The learning curve of the sign language recognition system

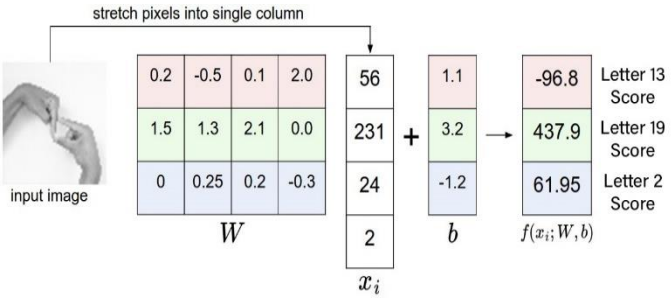


Fig. 11. Analysis of Image Processing (Softmax Classifier)

Conclusion

We used computer vision and machine learning skills to develop a Bangla sign language gesture detection system in this project. We discovered that sometimes simple solutions are more effective than complex solutions. Despite using a sophisticated segmentation technique, the most effective skin masks were extracted using a simple skin segmentation model. We also recognized the time restrictions and challenges of building a dataset from the ground up. Looking back, having a dataset to work with would have been preferable. Some letters were more difficult to classify in our live demonstration, such as "ঙ." Even though our classification approach works very well, as evidenced by the tables and photographs, there is still much room for future research.

Extending the gesture recognition system to all BDSL alphabets and additional non-alphabet gestures would be possible extensions to this project. We believe that by coding in C, we can boost the speed of our system, having chosen MATLAB as the implementation platform. This project's foundation can be used for additional applications, such as commanding robot navigation with hand gestures.

ACKNOWLEDGMENT

We want to thank the Department of Electrical and Computer Engineering of North South University, Bangladesh.

REFERENCES

[1] Khalifa, A. (2021, November 27). The Benefits of Learning Sign Language. Hear Me Out! [CC]. https://hearmeoutcc.com/benefits-learning-sign-language/

[2] Deafness and hearing loss. (n.d.). Www.Who.Int. Retrieved January 12, 2022, from https://www.who.int/news-room/fact-sheets/detail/deafness-and-hearing-loss

[3] M. T. Hoque, M. Rifat-Ut-Tauwab, M. F. Kabir, F. Sarker, M. N. Huda, and K. Abdullah-Al-Mamun, "Automated Bangla sign language translation system: Prospects, limitations and applications," 2016 5th International Conference on Informatics, Electronics and Vision (ICIEV), 2016, pp. 856-862, https://doi.org/10.1109/ICIEV.2016.7760123

[4] Disabilities. (2011). Www.Who.Int. https://www.who.int/disabilities/world_report/2011/report/National Association of the Deaf - NAD. (n.d.). Www.Nad.Org. https://www.nad.org/resources/american-sign-language/

[5] Jachak, R. (2021, December 14). American Sign Language Recognition - Towards Data Science. Medium. https://towardsdatascience.com/american-sign-language-recognition-using-cnn-36910b86d651

[6] J. Lee, K. -R. Lee, U. Ha, J. -H. Kim, K. Lee and H. -J. Yoo, "A 0.8V 82.9μW In-Ear BCI Controller System with 8.8 PEF EEG Instrumentational Amplifier and Wireless BAN Transceiver," 2018 IEEE Symposium on VLSI Circuits, 2018, pp. 123-124, https://doi.org/10.1109/VLSIC.2018.8502263

[7] Deaf and Blind Support. https://cdd.org.bd/deaf-blind-support/

[8] Watson, M. M., & Greenberg, B. R. (1988). Referential Communication Abilities of Learning-Disabled, Language-Learning-Disabled, and Normal School-Age Children. Perceptual and Motor Skills, 66(1), 11–18. https://doi.org/10.2466/pms.1988.66.1.11

[9] Grosvald, M., Lachaud, C., & Corina, D. (2012). Handshape monitoring: Evaluation of linguistic and perceptual factors in the processing of American Sign Language. Language and Cognitive Processes, 27(1), 117–141. https://doi.org/10.1080/01690965.2010.549667

[10] Cartmill, E. A., Beilock, S., & Goldin-Meadow, S. (2012). A word in the hand: Action, gesture, and mental representation in humans and nonhuman primates. Philosophical Transactions of the Royal Society B: Biological Sciences, 367(1585), 129–143. https://doi.org/10.1098/rstb.2011.0162

[11] Golam Kayas, Sakhawat Hossain, and Himeland MahrajHasan, "Automatic Recognition of Bangla Sign Language

[12] Using Artificial Neural Networks (ANNS) For Deaf And Dumb to Bridge The Communication Gap," International

[13] Journal of Recent Advances in Multidisciplinary Research, vol. 3, no. 7, pp.1649-1654, 2016. Cartmill, E. A., Beilock, S., & Goldin-Meadow, S. (2012). A word in the hand: Action, gesture, and mental representation in humans and nonhuman primates. Philosophical Transactions of the Royal Society B: Biological Sciences, 367(1585), 129–143. https://doi.org/10.1098/rstb.2011.0162

[14] Hearing loss - how it affects people - Better Health Channel. (n.d.). Www.Betterhealth.Vic.Gov.Au. https://www.betterhealth.vic.gov.au/health/conditionsandtreatments/hearing-loss-how-it-affects-people

[15] Kwolek, B., Baczynski, W., & Sako, S. (2021). Recognition of JSL fingerspelling using Deep Convolutional Neural Networks. Neurocomputing, 456, 586–598. https://doi.org/10.1016/j.neucom.2021.03.133

[16] Lei, X., Pan, H., & Huang, X. (2019). A Dilated CNN Model for Image Classification. IEEE Access, 7, 124087–124095. https://doi.org/10.1109/access.2019.2927169

[17] R. Shahriar, A. G. M. Zaman, T. Ahmed, S. M. Khan, and H. M. Maruf, "A communication platform between Bangla and sign language," 2017 IEEE Region 10 Humanitarian Technology Conference (R10-HTC), 2017, pp. 1-4, https://doi.org/10.1109/R10-HTC.2017.8288892

[18] CS231n Convolutional Neural Networks for Visual Recognition. (n.d.). CS231n Convolutional Neural Networks for Visual Recognition. Retrieved January 12, 2022, from https://cs231n.github.io/linear-classify/

[19] M. Sanzidul Islam, S. Sultana Sharmin Mousumi, N. A. Jessan, A. Shahariar Azad Rabby, and S. Akhter Hossain, "Ishara-Lipi: The First Complete MultipurposeOpen Access Dataset of Isolated Characters for Bangla Sign Language," 2018 International Conference on Bangla Speech and Language Processing (ICBSLP), Sylhet, 2018, pp. 1-4. https://doi.org/10.1109/ICBSLP.2018.8554466

[20] S. Sarkar. Segmentation-robust representations, matching, and modeling for sign language recognition, June 2011. CVPR Workshop on Gesture Recognition, Co-authors: Barbara Loeding, Ruiduo Yang, Sunita Nayak, Ayush Para.

[21] D. Metaxas. Sign language and human activity recognition, June 2011. CVPR Workshop on Gesture Recognition.

[22] P. Y. Simard. Best practices for convolutional neural networks applied to visual document analysis, August 2003. Seventh International Conference on Document Analysis and Recognition.

[23] CS231n Convolutional Neural Networks for Visual Recognition. (n.d.). CS231n Convolutional Neural Networks for Visual Recognition. Retrieved January 12, 2022, from https://cs231n.github.io/linear-classify